2026-7-15 Linux进程--程序地址空间

- 1. 程序地址空间
  - 1.1 从布局图到真实地址
    - 1.1.1 布局图描述的是进程眼中的内存
    - 1.1.2 用变量地址验证各区域的布局
    - 1.1.3 为什么说“程序地址空间”不够准确
  - 1.2 同一个地址为什么能有两个值
    - 1.2.1 fork() 后首先看到相同地址
    - 1.2.2 子进程修改数据后，地址仍然相同
    - 1.2.3 虚拟地址、页表与写时拷贝
  - 1.3 操作系统如何描述地址空间
    - 1.3.1 task_struct 与 mm_struct
    - 1.3.2 VMA：一段连续且权限一致的虚拟区域
    - 1.3.3 堆和栈变化时，VMA 如何调整
  - 1.4 从虚拟地址到物理内存
    - 1.4.1 页表项记录了什么
    - 1.4.2 MMU 负责查表，缺页由操作系统处理
    - 1.4.3 回看进程控制中的 fork() 与 exec
  - 1.5 为什么需要虚拟地址空间
    - 1.5.1 隔离进程并保护内核
    - 1.5.2 让进程布局稳定，让物理内存可以分散
    - 1.5.3 延迟分配，节省物理内存
  - 1.6 小结

1. 程序地址空间

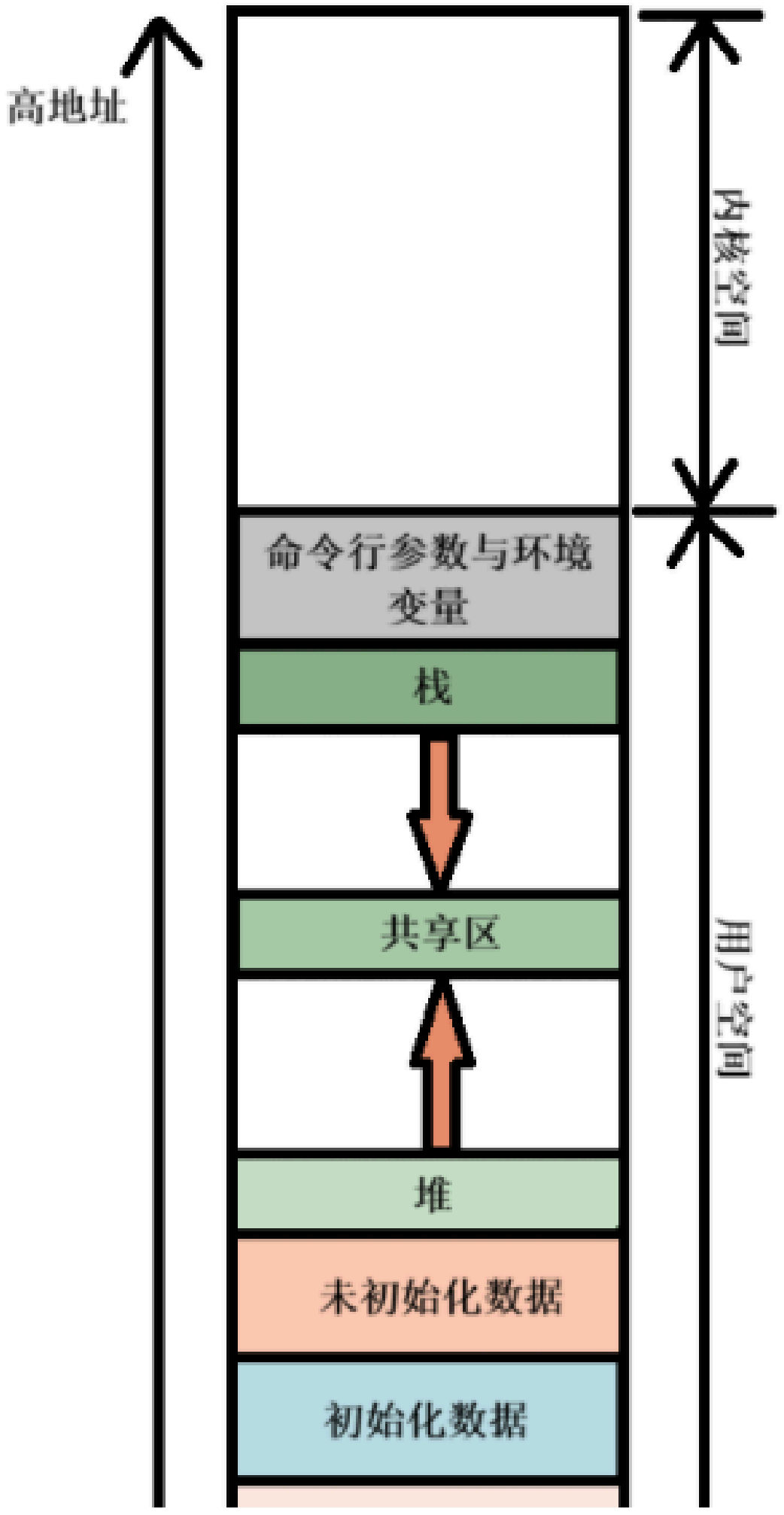
程序地址空间不是一块连续的物理内存，而是进程“看见”内存的方式。程序中的指针保存的是虚拟地址；CPU 访问数据时，先借助页表把虚拟地址翻译成物理地址，再访问真正的物理内存。

下面的内容沿着一条问题链展开：先确认布局图到底是什么，再用代码观察各区域，接着用 `fork()` 后“同址异值”的现象引出虚拟地址，最后解释操作系统如何描述、映射和管理这份地址空间。

1.1 从布局图到真实地址

1.1.1 布局图描述的是进程眼中的内存

学习 C 语言时常见的布局图如下：


高地址
内核空间
用户空间
命令行参数与环境变量
栈
共享区
堆
未初始化数据
初始化数据



图中的低地址到高地址依次包含正文代码、已初始化数据、未初始化数据、堆、共享区、栈，以及命令行参数和环境变量；再往上是内核空间。

这张图描述的不是物理内存的固定排布，而是进程的用户空间布局，也就是进程地址空间或虚拟地址空间。不同系统、编译器架构和链接方式会让具体地址范围发生变化，因此不应把图中某个固定数值当成所有平台都必须满足的结论。

用户态程序不能像访问普通变量一样直接访问内核空间。用户空间与内核空间的分界，以及各区域的具体权限，由操作系统和硬件共同维护。

1.1.2 用变量地址验证各区域的布局

可以用不同类型的数据观察它们在虚拟地址空间中的位置：

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int g_unval;            /* 未初始化全局变量：BSS */
int g_val = 100;        /* 已初始化全局变量：数据段 */

int main(int argc, char *argv[], char *env[])
{
    const char *str = "helloworld";
    static int test = 10;

    char *heap_mem  = (char *)malloc(10);
    char *heap_mem1 = (char *)malloc(10);
    char *heap_mem2 = (char *)malloc(10);
    char *heap_mem3 = (char *)malloc(10);

    printf("code addr: %p\n", (void *)main);
    printf("init global addr: %p\n", (void *)&g_val);
    printf("uninit global addr: %p\n", (void *)&g_unval);
    printf("test static addr: %p\n", (void *)&test);

    printf("heap addr: %p\n", (void *)heap_mem);
    printf("heap addr: %p\n", (void *)heap_mem1);
    printf("heap addr: %p\n", (void *)heap_mem2);
```

```
    printf("heap addr: %p\n", (void *)heap_mem3);

    printf("stack addr: %p\n", (void *)&heap_mem);
    printf("stack addr: %p\n", (void *)&heap_mem1);
    printf("stack addr: %p\n", (void *)&heap_mem2);
    printf("stack addr: %p\n", (void *)&heap_mem3);

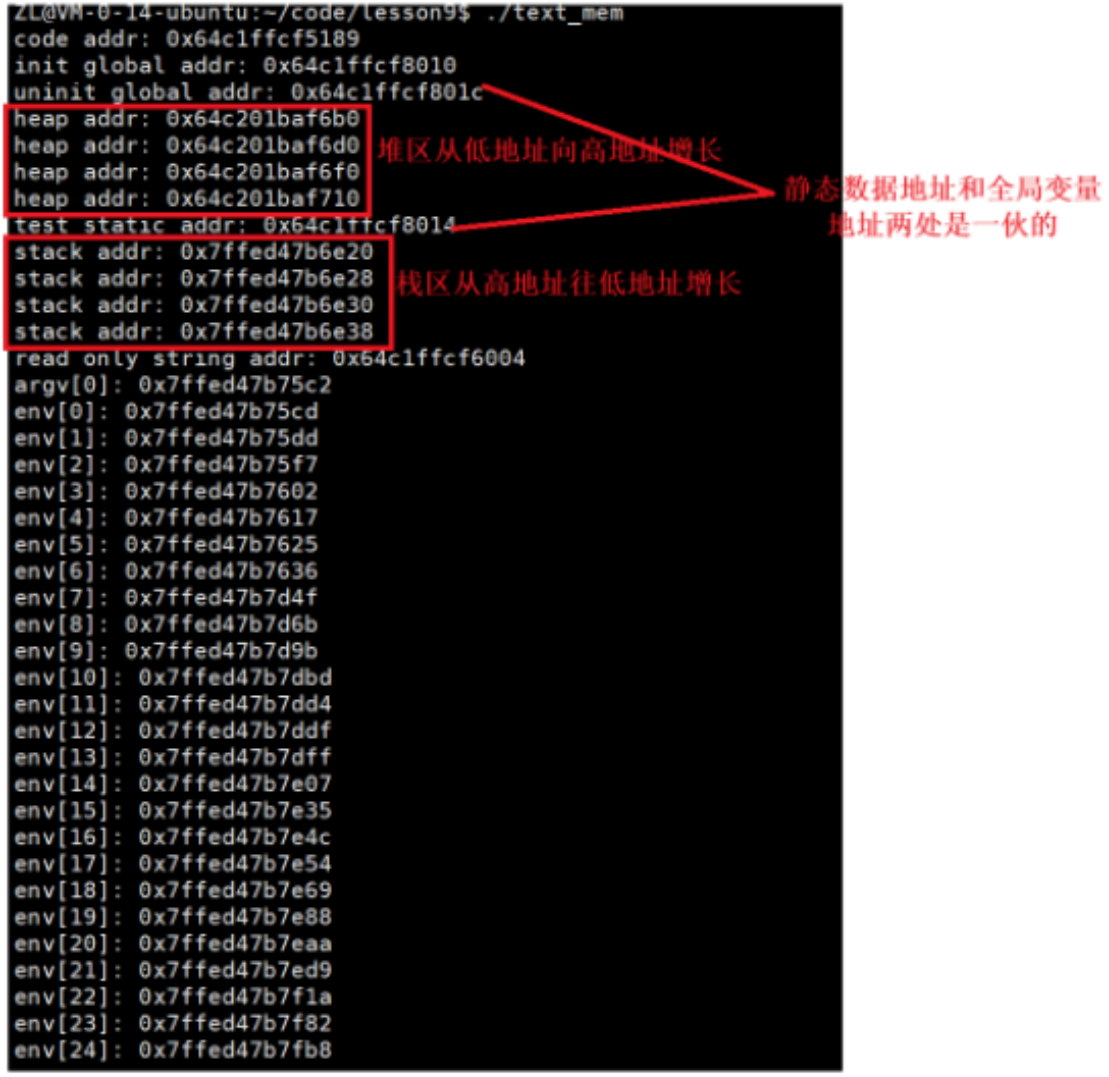
    printf("read only string addr: %p\n", (const void *)str);

    for (int i = 0; i < argc; i++) {
        printf("argv[%d]: %p\n", i, (void *)argv[i]);
    }

    for (int i = 0; env[i]; i++) {
        printf("env[%d]: %p\n", i, (void *)env[i]);
    }

    return 0;
}
```

一次运行结果如下：



可以把观察结果整理为：

数据或区域	典型位置	观察重点
函数、指令	正文代码段	通常只读、可执行
已初始化全局变量、静态变量	数据段	生命周期随进程存在
未初始化全局变量	BSS 段	不把大量零值直接写入可执行文件
`malloc()` 返回的内存	堆	常见实现中整体向高地址扩展
动态库、文件映射、共享内存	共享映射区	位于堆与栈之间，具体布局因平台而异
局部变量、函数参数	栈	常见实现中向低地址扩展
字符串字面量	只读数据区	内容通常不允许修改
命令行参数、环境变量	初始栈的高地址一侧	字符串本身通常位于这一区域

堆和栈并不是从两端“互相抢同一块物理内存”，而是两个具有不同用途、增长方向和权限的虚拟区域。它们的边界由内核调整，真正落到哪些物理页上则由页表决定。

还需要区分“指针变量本身”和“指针指向的内容”。例如：

- `str` 的值指向只读区的字符串 `"helloworld"`；
- `&str` 是这个指针变量自身的地址，通常位于栈上。

程序每次运行得到的绝对地址可能不同，这通常与地址随机化（ASLR）和程序加载基址有关。比较布局时，应看各区域的相对关系和地址变化方向，不能要求两次运行必须出现完全相同的绝对地址。

1.1.3 为什么说“程序地址空间”不够准确

“程序地址空间”容易让人以为某个可执行文件本身拥有这份空间。实际上，地址空间是在程序运行、成为进程后，由内核为进程建立并维护的。

更准确的说法是“进程地址空间”。同一份程序被启动多次时，会形成多个进程，每个普通进程都可以拥有自己的地址空间和页表。

1.2 同一个地址为什么能有两个值

1.2.1 `fork()` 后首先看到相同地址

先观察父子进程都不修改全局变量时的情况：

```
#include <stdio.h>
#include <unistd.h>

int g_val = 100;

int main(void)
{
    printf("g_val: %d, &g_val: %p\n", g_val, (void *)&g_val);

    pid_t id = fork();
    if (id < 0) {
        perror("fork");
        return 1;
    } else if (id == 0) {
        printf("child[%d]: %d : %p\n",
               getpid(), g_val, (void *)&g_val);
    } else {
```

```
        printf("parent[%d]: %d : %p\n",
               getpid(), g_val, (void *)&g_val);
    }

    sleep(1);
    return 0;
}
```

运行结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson9$ ./test_addr
g_val: 100,&g_val: 0x598f8e8e4010
parent[3847806]: 100 : 0x598f8e8e4010
child[3847807]: 100 : 0x598f8e8e4010
ZL@VM-0-14-ubuntu:~/code/lesson9$
```

父子进程的 PID 不同，但 `&g_val` 和变量值相同。子进程以父进程为模板创建，初始时可以将它们理解为共享同一份只读数据映射。

1.2.2 子进程修改数据后，地址仍然相同

让子进程修改 `g_val`：

```
#include <stdio.h>
#include <unistd.h>

int g_val = 0;

int main(void)
{
    printf("g_val: %d, &g_val: %p\n", g_val, (void *)&g_val);

    pid_t id = fork();
    if (id < 0) {
        perror("fork");
        return 1;
    } else if (id == 0) {
        g_val = 100;
        printf("child[%d]: %d : %p\n",
               getpid(), g_val, (void *)&g_val);
    } else {
        sleep(3);
```

```
        printf("parent[%d]: %d : %p\n",
               getpid(), g_val, (void *)&g_val);
    }

    sleep(1);
    return 0;
}
```

运行结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson9$ ./test_addr
g_val: 0,&g_val: 0x60f012248014
child[3849195]: 100 : 0x60f012248014
parent[3849194]: 0 : 0x60f012248014
ZL@VM-0-14-ubuntu:~/code/lesson9$
```

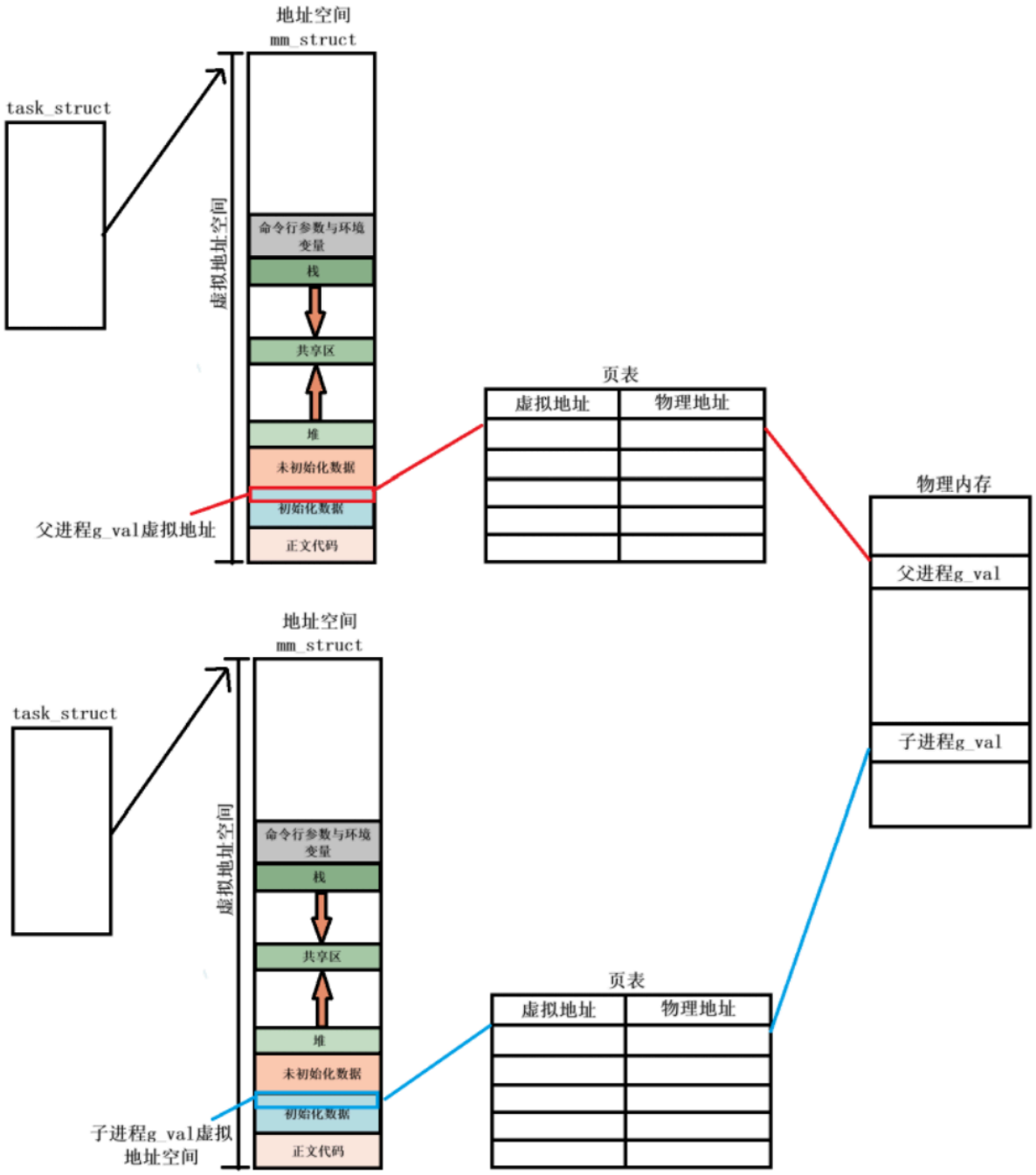
子进程读到 `100`，父进程仍然读到 `0`，但两边打印出的 `&g_val` 完全相同。

这不是“同一个物理地址同时保存了两个值”。父子进程打印的是各自的虚拟地址；两个相同的虚拟地址经过各自的页表，可以映射到不同的物理页。

1.2.3 虚拟地址、页表与写时拷贝

用户程序看到的地址是虚拟地址，不是物理内存地址。一次访问大致经历以下过程：

1. 程序使用虚拟地址访问变量；
2. MMU 根据当前进程的页表翻译地址；
3. 得到物理地址后，CPU 才访问对应的物理页；
4. 若页面不存在或权限不满足，就触发异常，由操作系统处理。



`fork()` 创建子进程时，没有必要立刻把所有数据复制一遍。父子进程可以先让对应的虚拟页指向相同的物理页，并把共享页面标记为写保护，这种做法类似“浅拷贝”。

当任意一方开始写入时，内核才真正复制这一页：

1. 为该进程分配新的物理页；
2. 把原页面内容复制过去；
3. 修改该进程的页表映射；
4. 保持虚拟地址不变，重新执行写操作。

这就是**写时拷贝**（Copy-on-Write，COW）。

因此，`fork()` 后常见的结论是：

- 代码通常可以继续共享；
- 未被修改的数据可以共享；
- 一旦写入，数据页就按需复制；
- 父子进程的虚拟地址可以相同，但物理页可以不同；
- 进程之间的数据修改不会通过普通变量互相“串过去”。

物理地址被操作系统隐藏起来，并不是为了制造神秘感，而是为了访问控制和数据保护。如果用户态程序能直接用指针访问任意物理地址，进程隔离和内核安全就无从谈起。

1.3 操作系统如何描述地址空间

1.3.1 `task_struct` 与 `mm_struct`

Linux 用 `task_struct` 描述一个执行任务，用 `mm_struct` 描述它的用户地址空间。`task_struct` 中保存着指向 `mm_struct` 的指针：

```
struct task_struct {
    /* ... */
    struct mm_struct *mm;
    struct mm_struct *active_mm;
    /* ... */
};
```

对于普通用户进程，`mm` 指向该进程使用的地址空间。内核线程通常没有自己的用户空间，因此 `mm` 为 `NULL`，可通过 `active_mm` 借用某个进程的内核映射。

`mm_struct` 不是一个连续的大数组，也不等于整块物理内存。它是一份内核元数据，用来描述这个用户地址空间由哪些区域组成、各区域的边界在哪里。

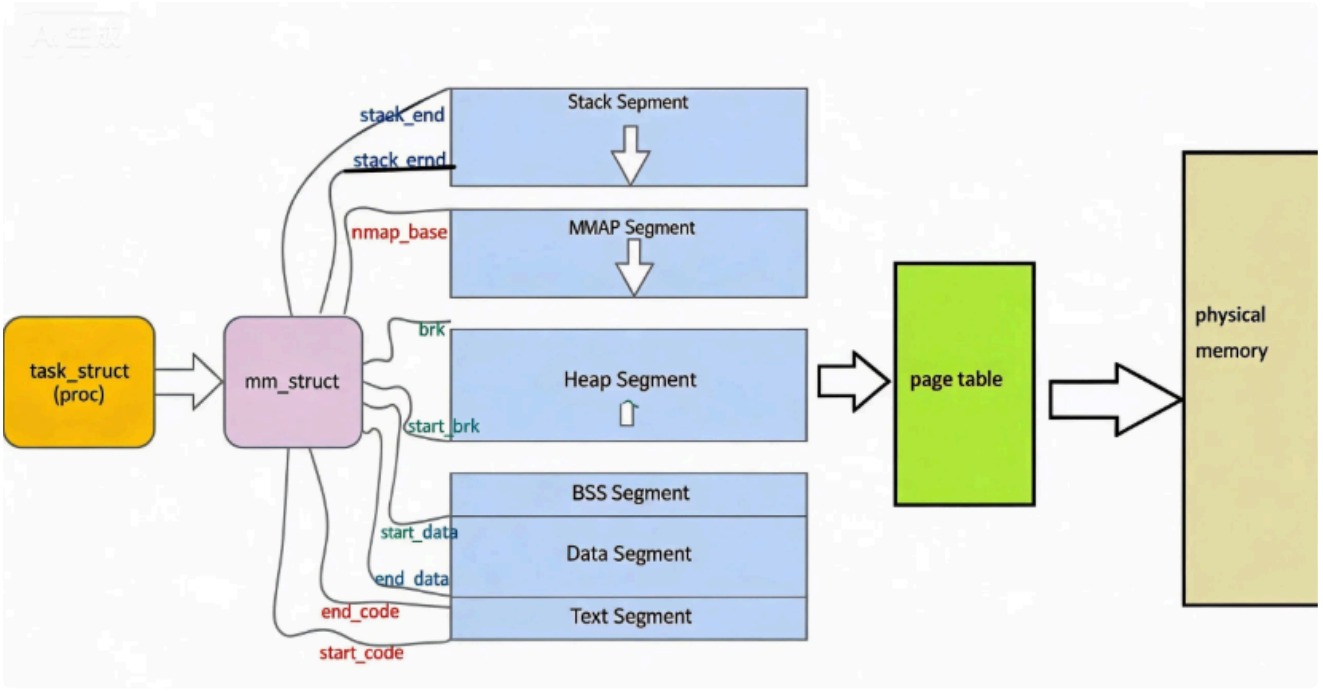
```
struct mm_struct {
    /* ... */
    struct vm_area_struct *mmap;     /* 指向 VMA 链表 */
    struct rb_root mm_rb;            /* 红黑树根 */
    unsigned long task_size;          /* 地址空间大小 */

    unsigned long start_code, end_code;
    unsigned long start_data, end_data;
```

```
    unsigned long start_brk, brk;
    unsigned long start_stack;
    unsigned long arg_start, arg_end;
    unsigned long env_start, env_end;
    /* ... */
};
```

这里的 `start_*`、`end_*` 描述了代码段、数据段等区域的边界；`brk` 表示堆区当前边界；`start_stack` 表示栈的起点。全局变量、静态变量的生命周期之所以跟随进程，是因为它们所属的虚拟区域一直保留到进程结束。

同一进程中的多个线程通常共享同一个 `mm_struct`，因此也共享代码、数据和堆；每个线程主要拥有自己的执行上下文和栈。不同普通进程一般拥有各自的 `mm_struct`，所以默认不能直接读写对方的数据。



1.3.2 VMA：一段连续且权限一致的虚拟区域

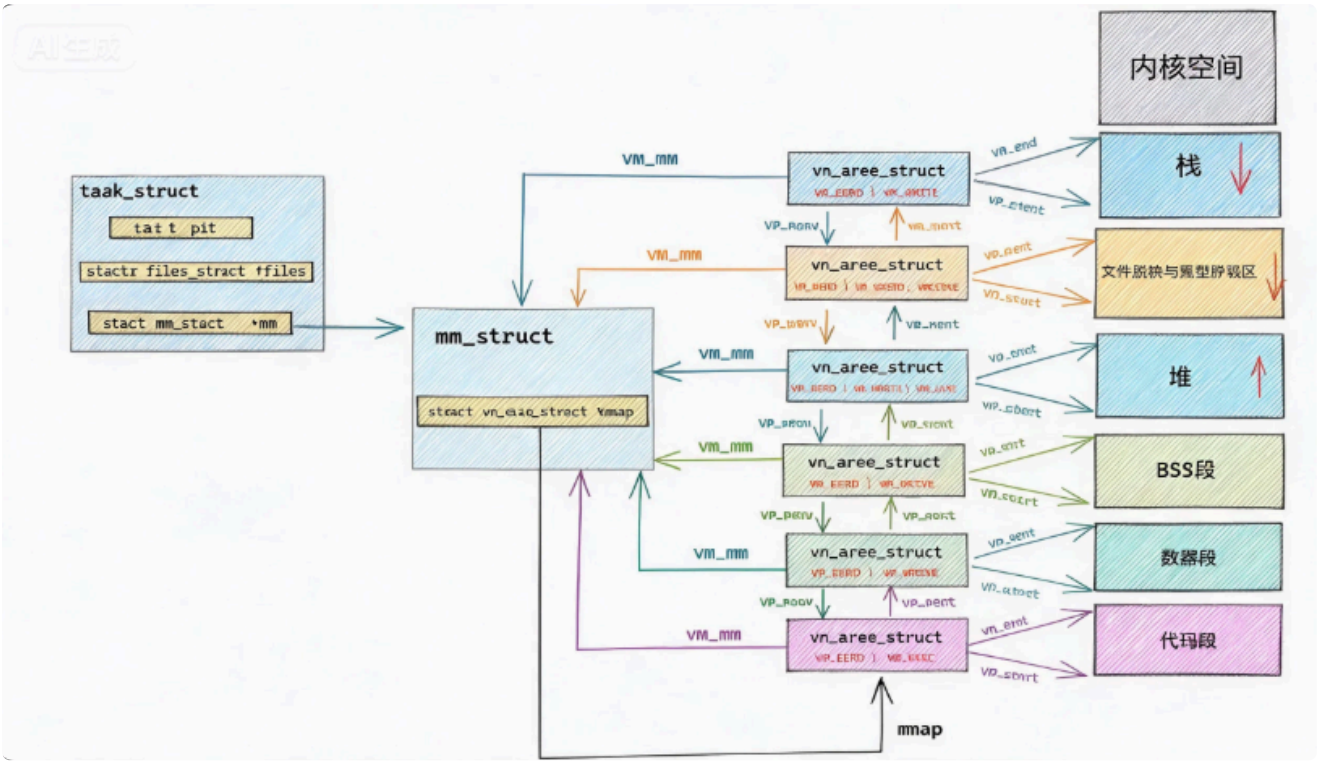
`mm_struct` 描述整体，`vm_area_struct` 则描述其中一个独立区域，也就是 VMA（Virtual Memory Area）。一个 VMA 通常是一段连续、用途和权限相同的虚拟地址范围。

```
struct vm_area_struct {
    unsigned long vm_start;       /* 区域起始地址 */
    unsigned long vm_end;         /* 区域结束地址 */
    struct vm_area_struct *vm_next;
    struct vm_area_struct *vm_prev;
    unsigned long vm_flags;       /* 读、写、执行等属性 */
```

```
    struct mm_struct *vm_mm;        /* 所属地址空间 */
    struct file *vm_file;           /* 文件映射来源 */
    unsigned long vm_pgoff;         /* 文件内偏移 */
    /* ... */
};
```

其中 `vm_start` 和 `vm_end` 限定地址范围，`vm_flags` 与页表权限共同约束能否读、写、执行。代码段和只读字符串不能被普通写操作修改，就是区域属性和实际页面权限共同作用的结果。

以经典 Linux 实现为例，VMA 同时可以通过链表依次遍历，也可以通过红黑树快速查找某个地址属于哪个区域。单链表便于顺序处理，红黑树便于在大地址空间中定位；它们不是两份互不相干的区域，而是对同一组 VMA 的两种组织方式。现代内核的具体数据结构可能继续演进，但“用 VMA 描述一段连续虚拟区域”这个核心思想不变。



1.3.3 堆和栈变化时，VMA 如何调整

布局图看起来方方正正，但堆和栈的范围会动态变化：

- 堆相关系统调用或分配器可以调整 `brk`，从而扩大或缩小堆区域；
- 函数调用、局部变量和返回地址会让栈向低地址方向增长；
- 两个区域扩到相邻时，可能合并；
- 区域中间被释放出空洞时，可能拆分；
- 动态库、匿名映射和文件映射也会创建额外 VMA。

区域边界的重新划分，本质上是更新 `vm_start`、`vm_end`、`brk` 等元数据，以及调整页表和 VMA 的组织结构。

`malloc()` 返回的是一段可用的虚拟地址，不保证每个字节此时都已有物理页。分配器可能先扩大虚拟堆区；真正访问某个新页面时，才通过缺页机制补上物理内存。大块分配也可能改用 `mmap()` 建立独立映射，这属于同一套虚拟地址管理思路。

1.4 从虚拟地址到物理内存

1.4.1 页表项记录了什么

虚拟地址空间和物理内存通常都按页划分，页表保存“虚拟页 → 物理页框”的映射。一个页表项（PTE）除了物理页框信息，还会记录权限和状态：

- Present/Valid：该映射当前是否可用；
- R/W：是否允许写入；
- User/Supervisor：用户态能否访问；
- Execute：是否允许执行；
- Accessed、Dirty：是否访问过、是否写过。

由于映射关系由操作系统维护，程序无法随意把某个虚拟地址接到别人正在使用的物理页上。同样的虚拟地址，在不同进程中也可以指向不同物理页。

1.4.2 MMU 负责查表，缺页由操作系统处理

CPU 产生的是虚拟地址，而实际访问内存需要物理地址。MMU（Memory Management Unit）负责地址转换，TLB 会缓存最近使用的页表结果以加快查询。

页表并不是由用户程序逐条手工查询的。地址翻译主要由硬件完成；当映射不存在、页面被换出或权限不满足时，硬件触发异常，操作系统再介入处理。

“页表项暂时不可用”不一定表示页面一定在磁盘上。它可能只是尚未分配物理页，也可能是写时拷贝造成的共享只读页。内核会根据异常原因区分：

- 按需分配一个新物理页；
- 为写时拷贝复制页面；
- 从交换区换回页面；
- 对越权访问发出进程异常。

因此，“虚拟地址没有立即对应物理页”正是实现按需分页的基础。

1.4.3 回看进程控制中的 `fork()` 与 `exec`

进程控制中的两个操作可以直接帮助理解地址空间：

- `fork()` 创建一个新进程。父子进程初始可以共享大量页面，写时再由页表映射把它们分开。
- `exec` 不创建新进程，而是把当前进程用户空间中的代码和数据替换成磁盘上的新程序，PID 保持不变。

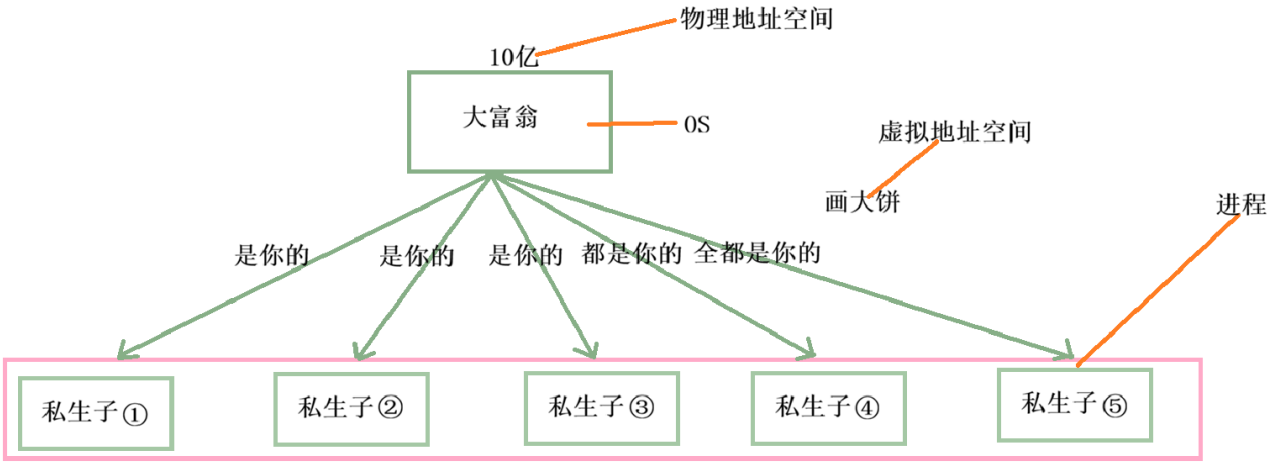
`exec` 的成功执行不再返回原程序，因为它已经换了一套用户空间内容。这也说明：进程身份由其任务控制信息、资源和执行状态等共同维持；代码与数据所在的地址空间内容可以被重新建立和替换。

所以，“进程”不能简单等同于“一段固定不变的代码加数据”。更合适的理解是：

> 进程 = 内核中的执行与资源管理信息 + 当前地址空间及其代码、数据 + 执行上下文。

1.5 为什么需要虚拟地址空间

虚拟地址空间最初是一个很抽象的概念，可以借“大富翁给每个孩子画出十亿元家产”的类比理解：每个进程都拥有一份看起来很完整、很独立的地址范围，但这不等于系统同时准备了同样多的物理内存。



更重要的是，这不是单纯“给每个进程画大饼”，而是一套完整的地址隔离和资源调度机制。

1.5.1 隔离进程并保护内核

每次访问都必须经过页表映射与权限检查。进程只能使用属于自己的虚拟区域，不能仅凭一个数字地址读写其他进程或内核的数据。

页表项中的读写、用户态/内核态和可执行权限，把“地址是否合法”“能否写”“能否执行”等规则落实到了硬件访问路径上。

1.5.2 让进程布局稳定，让物理内存可以分散

进程看到的是稳定的代码段、数据段、堆、栈等虚拟区域；这些区域对应的物理页却不必连续，也不必按固定顺序存放。

因此：

- 物理内存可以利用分散的页框；
- 同一程序每次仍能按一致的逻辑布局运行；
- 进程管理关注进程和资源，内存管理关注物理页分配，两者通过页表关联，降低耦合。

1.5.3 延迟分配，节省物理内存

创建进程或申请内存时，系统可以先只建立虚拟区域和页表，不必马上分配全部物理页。只有真正访问页面时才触发缺页处理，这就是按需分页或懒加载。

它带来几个直接好处：

- 进程启动更快，因为无需预先复制全部页面；
- 未使用的 `malloc()` 或 `new` 内存不会立刻占满物理内存；
- 多个进程可以共享只读代码页；
- 借助交换区，系统可以让程序使用的虚拟地址总量超过当前空闲物理内存。

1.6 小结

程序地址空间的核心可以归纳为一条链：

> 布局图是虚拟视图 → 代码中的地址是虚拟地址 → 每个进程通过自己的页表和 VMA 描述这份视图 → 虚拟页按需映射到物理页。

再回看 `fork()` 后“地址相同、值不同”的现象：

1. 两个进程的 `&g_val` 是相同的虚拟地址；
2. 子进程修改页面时发生写时拷贝；
3. 内核为写的一方建立新的物理页和新的页表映射；
4. 虚拟地址不变，但两份映射指向不同物理页，于是同一个虚拟地址可以读出不同值。

所以，进程地址空间既不是一整块物理内存，也不是程序可执行文件的简单副本。它是由内核描述的虚拟区域、由页表建立的地址映射，以及由硬件完成访问控制的一套组合机制。